

INGÉNIERIE DU LOGICIEL

Cours de préparation — S7 M1 Informatique (ILIA)

Du cycle de vie logiciel aux pratiques modernes de développement : architecture, gestion de projet, tests, qualité de code et intégration continue — avec exercices corrigés.

Sommaire

1. Cycle de vie logiciel
2. Architecture logicielle
3. Gestion de projet et exigences
4. Tests logiciels
5. Qualité de code
6. Gestion de version et CI/CD
7. Exercices récapitulatifs

Chapitre 1 — Cycle de vie logiciel

Un projet logiciel traverse plusieurs phases, de l'expression du besoin à la maintenance. La façon dont on organise ces phases définit le **modèle de cycle de vie** adopté.

1.1 Le modèle en cascade (Waterfall)

Chaque phase est réalisée entièrement avant de passer à la suivante, de façon linéaire et séquentielle.

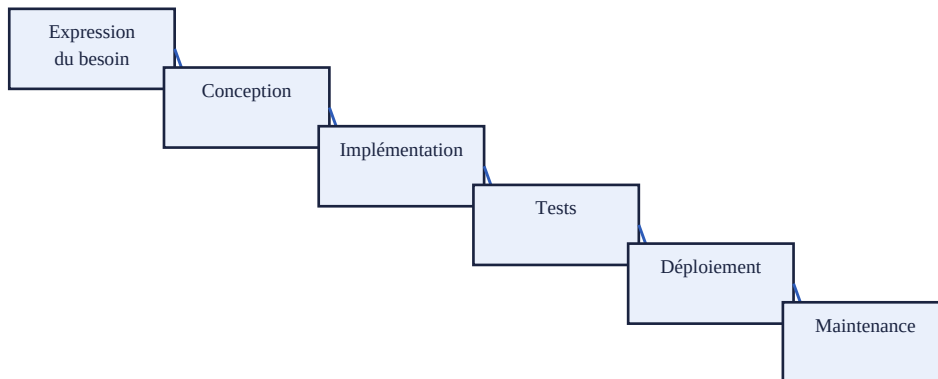


Figure 1.1 — Le modèle en cascade : phases séquentielles

- **Avantage** : structure claire, documentation exhaustive, adapté quand les besoins sont stables et bien connus dès le départ.
- **Inconvénient** : très rigide — un problème découvert tardivement (ex. en test) coûte cher à corriger car il faut remonter les phases précédentes.

1.2 Les méthodes agiles et Scrum

Les méthodes **agiles** répondent aux limites du modèle en cascade en découpant le développement en cycles courts et itératifs, avec des retours fréquents. **Scrum** en est le framework le plus répandu.

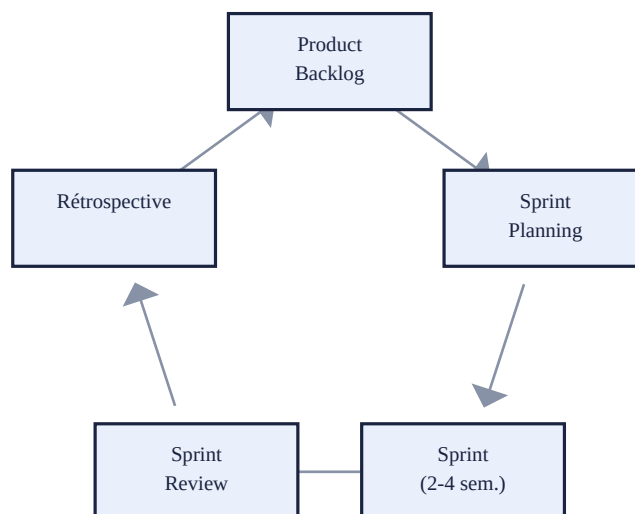


Figure 1.2 — Le cycle Scrum : backlog, planning, sprint, review, rétrospective

Rôle Scrum	Responsabilité
Product Owner	Priorise le backlog produit, représente les besoins métier
Scrum Master	Facilite le processus, retire les obstacles de l'équipe
Équipe de développement	Réalise le travail technique pendant le sprint

À retenir : un *sprint* dure typiquement 2 à 4 semaines et se termine par un incrément de logiciel potentiellement livrable, suivi d'une revue et d'une rétrospective.

Exercice 1 — Choisir un modèle de cycle de vie

Un client vous commande un logiciel de contrôle pour un satellite, avec un cahier des charges très précis et figé, validé par une autorité de certification avant tout développement. Quel modèle de cycle de vie privilégier, cascade ou agile ? Justifiez.

Correction

Le modèle en **cascade** est plus adapté ici : les exigences sont stables, exhaustivement définies et validées formellement en amont (contrainte de certification), ce qui correspond exactement au contexte où la cascade est pertinente — contrairement à un projet où les besoins évoluent en cours de route, terrain de prédilection de l'agile.

Chapitre 2 — Architecture logicielle

L'**architecture logicielle** définit comment un système est découpé en composants et comment ces composants interagissent. Un bon découpage facilite la maintenance, les tests et l'évolution du logiciel.

2.1 Architecture en couches

Le système est organisé en couches empilées (présentation, métier, accès aux données), chaque couche ne communiquant qu'avec ses voisines immédiates.

Couche	Rôle
Présentation	Interface utilisateur, affichage
Métier (business logic)	Règles et traitements propres au domaine
Accès aux données	Persistance, requêtes base de données

2.2 Le patron MVC (Modèle-Vue-Contrôleur)

MVC sépare une application interactive en trois responsabilités bien distinctes, ce qui facilite en particulier de faire évoluer l'affichage sans toucher à la logique métier.

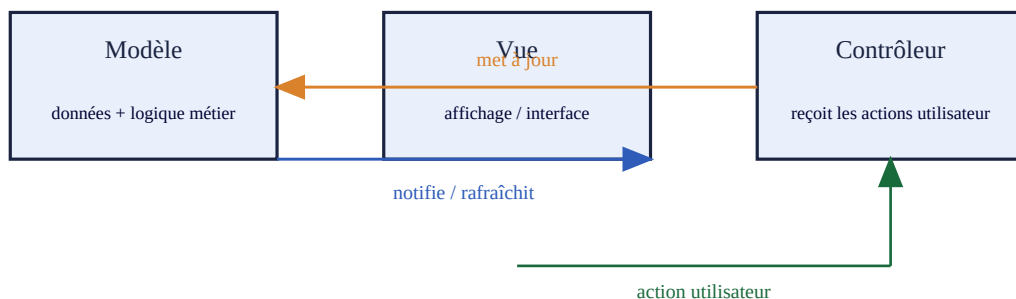


Figure 2.1 — Le patron MVC et le flux d'interactions entre ses trois composants

2.3 Microservices

Plutôt qu'une seule application monolithique, le système est découpé en petits services indépendants, déployables séparément, communiquant par API (souvent REST ou messages asynchrones).

Architecture	Avantage principal	Inconvénient principal
Monolithe	Simple à développer et déployer au départ	Difficile à faire évoluer / scaler à grande échelle
Microservices	Services déployés et scalés indépendamment	Complexité opérationnelle (réseau, cohérence des données)

Exercice 2 — Choisir une architecture

Une petite équipe de 3 développeurs démarre un nouveau produit ; le périmètre fonctionnel est encore incertain et va beaucoup évoluer dans les premiers mois. Monolithe ou microservices ? Justifiez.

Correction

Monolithe dans un premier temps : avec une petite équipe et un périmètre incertain, la complexité opérationnelle des microservices (déploiements multiples, communication réseau, cohérence des données) coûterait plus cher que ce qu'elle apporterait. Il est courant de démarrer en monolithe bien structuré (en couches, avec des modules bien séparés), puis de migrer vers des microservices si et quand le besoin de scalabilité ou d'autonomie d'équipes se fait vraiment sentir.

Chapitre 3 — Gestion de projet et exigences

Bien spécifier ce qu'on doit construire est aussi important que de savoir le construire. Ce chapitre couvre les outils courants pour capturer et organiser les besoins.

3.1 User stories

Une **user story** décrit un besoin du point de vue de l'utilisateur, selon un format standard :

```
En tant que <rôle>,  
Je veux <fonctionnalité>,  
Afin de <bénéfice attendu>.
```

Exemple : « En tant qu'utilisateur inscrit, je veux réinitialiser mon mot de passe, afin de retrouver l'accès à mon compte en cas d'oubli. »

3.2 Critères d'acceptation

Chaque user story est complétée par des **critères d'acceptation** précis, qui permettent de savoir objectivement quand la fonctionnalité est terminée. Un format courant est *Étant donné / Quand / Alors* (Given/When/Then).

```
Étant donné un utilisateur inscrit avec un email valide,  
Quand il clique sur « mot de passe oublié » et saisit son email,  
Alors un lien de réinitialisation lui est envoyé par email.
```

3.3 Exigences fonctionnelles vs non fonctionnelles

Type	Répond à la question	Exemple
Fonctionnelle	Que doit faire le système ?	Le système doit permettre de filtrer les commandes par date
Non fonctionnelle	Comment le système doit-il se comporter ?	Le temps de réponse doit rester inférieur à 200 ms pour 95% des requêtes

Exercice 3 — Écrire une user story

Un site e-commerce veut permettre à un client de suivre l'état de sa commande en temps réel. Rédigez la user story correspondante, puis un critère d'acceptation au format Given/When/Then.

Correction

User story : « En tant que client, je veux suivre l'état de ma commande en temps réel, afin de savoir quand je vais la recevoir. »

Critère d'acceptation : Étant donné une commande passée et en cours de livraison, quand le client consulte la page de suivi, alors le statut actuel de la commande (préparée, expédiée, livrée) s'affiche avec la date de mise à jour la plus récente.

Chapitre 4 — Tests logiciels

Tester un logiciel, c'est vérifier qu'il se comporte comme attendu. Les tests se déclinent en plusieurs niveaux, souvent représentés par une **pyramide des tests**.

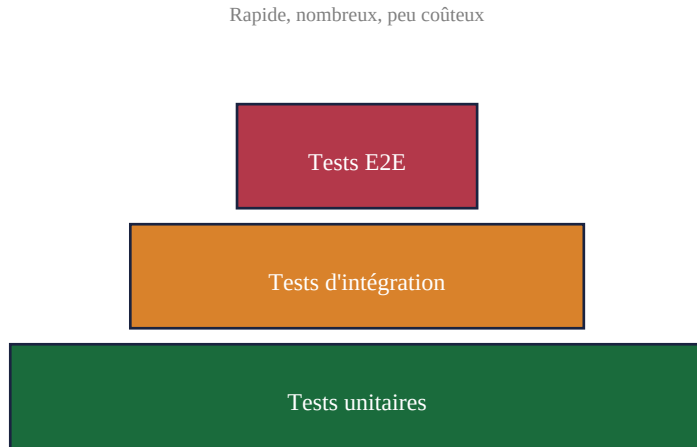


Figure 4.1 — Pyramide des tests : nombreux tests unitaires à la base, peu de tests E2E au sommet

Niveau	Portée	Vitesse
Unitaire	Une seule fonction/méthode/classe, isolée (mocks pour les dépendances)	Très rapide
Intégration	Plusieurs composants ensemble (ex. service + base de données)	Moyenne
End-to-end (E2E)	Le système complet, du point de vue de l'utilisateur final	Lente

4.1 TDD (Test-Driven Development)

Le TDD inverse l'ordre habituel : on écrit d'abord le test (qui échoue forcément), puis le code minimal pour le faire passer, puis on améliore (refactor) — cycle **Red / Green / Refactor**.

1. Red : écrire un test qui échoue pour une fonctionnalité pas encore implémentée
2. Green : écrire le code le plus simple possible pour faire passer ce test
3. Refactor : améliorer le code sans changer son comportement (les tests protègent la non-régression)

Exercice 4 — Niveau de test approprié

Pour chacun des cas suivants, indiquez le niveau de test le plus adapté (unitaire, intégration, E2E) :

- a) Vérifier qu'une méthode `calculerRemise(prix, pourcentage)` renvoie le bon résultat.
- b) Vérifier qu'un formulaire d'inscription complet fonctionne, de la saisie jusqu'à la réception de l'email de confirmation.
- c) Vérifier qu'un service qui écrit en base de données persiste correctement les données.

Correction

- a) Test unitaire — une seule fonction pure, sans dépendance externe.
- b) Test E2E — parcours utilisateur complet impliquant plusieurs systèmes (interface, backend, service d'email).
- c) Test d'intégration — vérifie l'interaction réelle entre le service et la base de données.

Chapitre 5 — Qualité de code

5.1 Dette technique

La **dette technique** désigne le coût futur généré par des choix de développement rapides mais non idéaux (raccourcis, duplication, absence de tests). Comme une dette financière, elle génère des « intérêts » : plus on attend pour la corriger, plus elle coûte cher à rembourser.

Exemple concret : copier-coller une fonction à trois endroits au lieu de la factoriser fait gagner du temps immédiatement, mais chaque futur bug ou évolution devra être répliqué trois fois — la dette technique s'accumule silencieusement.

5.2 Refactoring

Le **refactoring** consiste à améliorer la structure interne du code (lisibilité, duplication, complexité) **sans changer son comportement observable**. Des tests automatisés solides sont indispensables pour refactorer en confiance.

Exemple de refactoring : extraction de méthode

```
// Avant double total = 0; for (Produit p : panier) { total += p.getPrix() *  
p.getQuantite(); } // Après double total = calculerTotal(panier);
```

5.3 Revue de code

La **revue de code** (code review) consiste à faire relire ses modifications par un ou plusieurs collègues avant de les intégrer au code principal. Elle permet de détecter des erreurs, partager la connaissance du code, et maintenir une cohérence de style dans l'équipe.

Exercice 5 — Identifier de la dette technique

Un développeur, pressé par une deadline, désactive temporairement les tests automatiques qui échouent plutôt que de corriger le bug, pour pouvoir livrer à temps. Sur le long terme, pourquoi ce choix est-il risqué ? Que faudrait-il faire ensuite ?

Correction

Désactiver des tests qui échouent masque un vrai problème sans le résoudre : c'est de la dette technique pure, car le bug reste présent et pourrait affecter les utilisateurs, tout en donnant une fausse impression de sécurité (« les tests passent »). Il faudrait tracer explicitement cette dette (ticket, TODO documenté) et prévoir un temps dédié rapidement après la livraison pour réactiver le test et corriger le bug sous-jacent, plutôt que de laisser la désactivation devenir permanente par oubli.

Chapitre 6 — Gestion de version et CI/CD

6.1 Git et le travail en branches

Un système de gestion de version comme Git permet à plusieurs développeurs de travailler en parallèle sans s'écraser mutuellement, via des **branches** isolées, fusionnées ensuite dans la branche principale.

Commande	Usage
git checkout -b feature/x	Créer et basculer sur une nouvelle branche
git commit -m "..."	Enregistrer un ensemble de modifications
git merge / git rebase	Intégrer les modifications d'une branche dans une autre
git push / git pull	Synchroniser avec le dépôt distant

6.2 Intégration et déploiement continu (CI/CD)

L'**intégration continue** (CI) automatise la compilation et l'exécution des tests à chaque modification du code (souvent à chaque push ou pull request), pour détecter les régressions le plus tôt possible. Le **déploiement continu** (CD) automatise ensuite la mise en production des versions validées.

Pipeline CI/CD typique : push du code → compilation → tests automatisés → analyse de qualité de code → déploiement (environnement de test, puis production si tout est validé).

Exercice 6 — Pipeline CI/CD

Un développeur pousse une modification qui casse un test unitaire existant. Décrivez ce qui devrait se passer dans un pipeline CI/CD bien configuré, et pourquoi c'est préférable à une détection manuelle tardive.

Correction

Le pipeline CI devrait automatiquement lancer la suite de tests dès le push, détecter l'échec du test unitaire, et bloquer la fusion (merge) ou le déploiement de cette modification tant que le problème n'est pas corrigé — souvent avec une notification immédiate au développeur. C'est préférable à une détection manuelle tardive car l'erreur est découverte en quelques minutes, pendant que le contexte est encore frais en mémoire, plutôt que des jours plus tard en production où le coût de correction (et l'impact utilisateur potentiel) est bien plus élevé.

Chapitre 7 — Exercices récapitulatifs

Exercice 7 — Synthèse — du besoin au déploiement

Une équipe agile développe une fonctionnalité de panier d'achat pour un site e-commerce.

- a) Rédigez une user story pour « ajouter un produit au panier ».
- b) À quel niveau de la pyramide des tests placeriez-vous un test qui vérifie que le total du panier se recalcule correctement après ajout d'un produit (sans base de données, fonction pure) ?
- c) Une fois la fonctionnalité développée et testée, quelles étapes du pipeline CI/CD devraient s'exécuter avant la mise en production ?

Correction

- a) « En tant que client, je veux ajouter un produit à mon panier, afin de pouvoir l'acheter avec d'autres articles lors du paiement. »
- b) Test unitaire — c'est une fonction de calcul pure, isolée, sans dépendance externe.
- c) Compilation, exécution de la suite de tests automatisés (unitaires puis intégration), analyse de qualité de code, déploiement en environnement de test, puis déploiement en production une fois toutes les étapes validées.

Exercice 8 — Question ouverte

Expliquez pourquoi une architecture en couches bien respectée facilite l'écriture de tests unitaires efficaces.

Correction

Une architecture en couches bien respectée isole la logique métier de la couche de présentation et de la couche d'accès aux données. On peut donc tester la logique métier seule, en isolant ses dépendances (par exemple en remplaçant l'accès à la base de données par un mock), sans avoir besoin de démarrer une interface graphique ou une vraie base de données. Cela rend les tests unitaires rapides, fiables et faciles à écrire, exactement ce que recherche la base de la pyramide des tests.